

자연어처리1 12주차 과제

↗ 큰 계획	🔄 02. 자연어처리1
🔍 과목	자연어 처리1 1주차,자연어처리1 3주차,자연어처리1 4주차,자연어처리1 4주차 과제,자연어처리1 팀플 (제출용),자연어처리1 팀플 주제 발표 논의 ,자연어처리1 5주차,자연어처리1 중간고사,자연어처리1 팀플, 추가할 것,자연어처리1 6주차,자연어처리1 2주차,자연어처리1 7주차, 자연어처리1 9주차,자연어처리1 10주차,자연어처리1 11주차,자연어처리1 12주차,자연어처리1 13주차,자연어처리1 14주차,자연어처리1 기말고사,유튜브 스크립트 코드(GPT),유튜브 스크립트 데이터셋(캐글), 최인엽 교수님께 여쭙보기,자연어처리1 10주차 과제,자연어처리 팀플 중간 발표,train.py,Dynamic Context Gate Def,output,중간 발표 PPT 구성,자연어처리1 12주차 과제
🔍 📅 Study	02. 자연어처리1
📅 planned date	@2025년 5월 26일
☰ 카테고리	
☑ Finished	☐
Σ yesterday	☐
🔍 PP	0.55
☀ Completeness	Not started
↗ Parent item	<u>자연어처리1 12주차</u>
☰ landin	
👤 with?	

자연어처리 12주차 실습 과제

이름	학번
이민우	202204249
류동우	202204046

1. TimeLSTM class의 forward/backward 소스 코드 분석 (총 2점)

- forward() 함수 코드 한줄 한줄을 분석 (6장, p264) (1점)

```
def forward(self, xs):
    Wx, Wh, b = self.params
    N, T, D = xs.shape
    H = Wh.shape[0]

    self.layers = []
    hs = np.empty((N, T, H), dtype='f')

    if not self.stateful or self.h is None:
        self.h = np.zeros((N, H), dtype='f')
    if not self.stateful or self.c is None:
        self.c = np.zeros((N, H), dtype='f')

    for t in range(T):
        layer = LSTM(*self.params)
        self.h, self.c = layer.forward(xs[:, t, :], self.h, self.c)
        hs[:, t, :] = self.h

        self.layers.append(layer)

    return hs
```

한 줄씩 상세 분석:

1줄: `Wx, Wh, b = self.params`

- 클래스 초기화 시 저장된 파라미터들을 언패킹
- Wx: 입력-은닉 가중치 행렬 ($D \times 4H$ 크기)
- Wh: 은닉-은닉 가중치 행렬 ($H \times 4H$ 크기)
- b: 편향 벡터 (4H 크기)

2줄: `N, T, D = xs.shape`

- 입력 데이터 xs의 차원 정보 추출
- N: 배치 크기 (미니배치의 샘플 수)
- T: 시계열 길이 (타임스텝 수)
- D: 입력 벡터의 차원 수

3줄: `H = Wh.shape[0]`

- 은닉 상태의 차원 수 H를 Wh 행렬의 첫 번째 차원에서 추출
- LSTM에서 은닉 상태와 셀 상태 모두 H 차원을 가짐

4줄: `self.layers = []`

- 각 타임스텝의 LSTM 레이어를 저장할 빈 리스트 초기화
- 역전파 시 각 타임스텝의 그래디언트 계산을 위해 필요

5줄: `hs = np.empty((N, T, H), dtype='f')`

- 모든 타임스텝의 은닉 상태를 저장할 3차원 배열 생성
- (배치크기, 타임스텝, 은닉차원) 형태의 빈 배열

6-7줄: 은닉 상태 초기화 조건문

```
if not self.stateful or self.h is None:
    self.h = np.zeros((N, H), dtype='f')
```

- `stateful=False` 이거나 이전 은닉 상태가 없으면 영벡터로 초기화
- `stateful=True` 면 이전 배치의 마지막 은닉 상태를 유지

8-9줄: 셀 상태 초기화 조건문

```
if not self.stateful or self.c is None:
    self.c = np.zeros((N, H), dtype='f')
```

- 셀 상태도 동일한 조건으로 초기화
- LSTM의 장기 기억을 담당하는 셀 상태 c

10줄: `for t in range(T):`

- 각 타임스텝에 대해 순차적으로 LSTM 연산 수행
- t=0부터 T-1까지 시간 순서대로 처리

11줄: `layer = LSTM(*self.params)`

- 현재 타임스텝을 위한 새로운 LSTM 레이어 인스턴스 생성
- 모든 타임스텝이 동일한 파라미터 (Wx, Wh, b)를 공유

12줄: `self.h, self.c = layer.forward(xs[:, t, :], self.h, self.c)`

- 현재 타임스텝 t의 입력 `xs[:, t, :]`와 이전 상태들로 LSTM 순전파 실행

13줄: `hs[:, t, :] = self.h`

- 현재 타임스텝의 은닉 상태를 결과 배열 hs에 저장

- 새로운 은닉 상태 h 와 셀 상태 c 를 반환 받아 업데이트

- 모든 타임스텝의 은닉 상태를 기록하여 최종 출력으로 사용

14줄: `self.layers.append(layer)`

- 현재 타임스텝의 LSTM 레이어를 리스트에 추가
- 역전파 시 각 레이어의 그래디언트 계산을 위해 보관

15줄: `return hs`

- 모든 타임스텝의 은닉 상태들을 담은 3차원 배열 반환
- 형태: (N, T, H)

- backward() 함수 코드 한줄 한줄을 분석 (6장, p264) (1점)

```
def backward(self, dhs):
    Wx, Wh, b = self.params
    N, T, H = dhs.shape
    D = Wx.shape[0]

    dxs = np.empty((N, T, D), dtype='f')
    dh, dc = 0, 0

    grads = [0, 0, 0]
    for t in reversed(range(T)):
        layer = self.layers[t]
        dx, dh, dc = layer.backward(dhs[:, t, :] + dh, dc)
        dxs[:, t, :] = dx
        for i, grad in enumerate(layer.grads):
            grads[i] += grad

    for i, grad in enumerate(grads):
        self.grads[i][...] = grad
    self.dh = dh
    return dxs
```

1줄: `Wx, Wh, b = self.params`

- forward와 동일하게 파라미터들을 언패킹

2줄: `N, T, H = dhs.shape`

- 상위 레이어에서 전달받은 그래디언트 dhs 의 차원 정보 추출

- 역전파에서도 동일한 파라미터 정보가 필요

3줄: `D = Wx.shape[0]`

- 입력 차원 D를 Wx 행렬의 첫 번째 차원에서 추출
- 입력에 대한 그래디언트 dxs의 크기 결정을 위해 필요

5줄: `dh, dc = 0, 0`

- 다음 타임스텝(미래)에서 전달받을 은닉/셀 상태 그래디언트 초기화
- 역전파는 미래에서 과거로 진행되므로 마지막 타임스텝부터 시작

7줄: `for t in reversed(range(T)):`

- 타임스텝을 역순으로 순회 (T-1, T-2, ..., 1, 0)
- 역전파는 시간의 역방향으로 진행되어야 함

9줄: `dx, dh, dc = layer.backward(dhs[:, t, :] + dh, dc)`

- 현재 타임스텝의 LSTM 레이어에서 역전파 수행
- 입력: 상위에서 온 그래디언트 + 미래 타임스텝에서 온 그래디언트
- 출력: 입력 그래디언트 dx, 이전 은닉상태 그래디언트 dh, 이전 셀상태 그래디언트 dc

11-12줄: 파라미터 그래디언트 누적

```
for i, grad in enumerate(layer.grads):
    grads[i] += grad
```

- dhs: 각 타임스텝 은닉 상태에 대한 그래디언트

4줄: `dxs = np.empty((N, T, D), dtype='f')`

- 각 타임스텝 입력에 대한 그래디언트를 저장할 3차원 배열 생성
- 최종적으로 하위 레이어로 전달할 그래디언트

6줄: `grads = [0, 0, 0]`

- 파라미터 그래디언트들을 누적할 리스트 초기화
- [dWx, dWh, db] 순서로 저장될 예정

8줄: `layer = self.layers[t]`

- 순전파 시 저장해둔 t번째 타임스텝의 LSTM 레이어 가져오기
- 해당 레이어의 중간 계산 결과들이 역전파에 필요

10줄: `dxs[:, t, :] = dx`

- 현재 타임스텝의 입력 그래디언트를 결과 배열에 저장
- 하위 레이어로 전달할 그래디언트 구성

13-14줄: 최종 그래디언트 저장

```
for i, grad in enumerate(grads):
    self.grads[i][...] = grad
```

- 각 타임스텝에서 계산된 파라미터 그래디언트를 누적
- 모든 타임스텝이 동일한 파라미터를 공유하므로 그래디언트를 합산
- 누적된 파라미터 그래디언트를 클래스의 `grads` 속성에 저장
- [...] 사용으로 기존 배열의 메모리 위치에 값 복사

15줄: `self.dh = dh`

- 첫 번째 타임스텝에서 나온 은닉 상태 그래디언트 저장
- stateful 모드에서 이전 배치와의 연결을 위해 보관

16줄: `return dxs`

- 모든 타임스텝의 입력 그래디언트를 하위 레이어로 반환
- 형태: (N, T, D)

@이민우

2. Rnnlm class 소스 코드 분석(6장, p266~268) (총 2점) @BBee

- 클래스에서 나오는 함수들의 기능을 설명 (코드 한줄 한줄 분석은 필요 없음)
- 클래스의 코드를 따라 전체적인 flow 설명

전체적인 Flow

Rnnlm은 LSTM 기반 언어 모델로, 단어 임베딩 → LSTM 처리 → 출력 변환 의 3단계 파이프라인을 구성한다. 학습 시에는 순전파로 손실을 계산하고, 역전파로 가중치를 업데이트하는 구조.

주요 함수 기능

`init ()` : 모델 초기화

임베딩, LSTM, 출력층의 가중치를 Xavier 초기화로 설정

TimeEmbedding, TimeLSTM, TimeAffine 계층을 순서대로 구성

모든 계층의 파라미터와 기울기를 통합 관리

`predict()` : 순전파 예측

입력 시퀀스를 각 계층에 순차적으로 통과시켜 다음 단어 확률 분포 출력

`forward()` : 학습용 순전파

`predict()` 로 점수를 구한 후 TimeSoftmaxWithLoss로 손실 계산

`backward()` : 역전파

손실층부터 시작해 각 계층을 역순으로 거슬러 올라가며 기울기 전파

`reset_state()` : LSTM 상태 초기화

새로운 시퀀스 처리 전 LSTM의 은닉 상태와 셀 상태를 리셋

3. RnnlmGen 클래스의 generate() 함수 코드 분석 (7장, p294~295) (1점) @BBee

- generate() 함수 코드 한줄 한줄을 분석

```
def generate(self, start_id, skip_ids=None, sample_size=100):
    word_ids = [start_id] # 시작 단어 ID로 결과 리스트 초기화

    x = start_id # 현재 입력 단어 설정
    while len(word_ids) < sample_size: # 원하는 길이까지 반복
        x = np.array(x).reshape(1, 1) # 배치와 시퀀스 차원 추가 (1x1 형태)
        score = self.predict(x) # 현재 단어로 다음 단어 확률 분포 예측
        p = softmax(score.flatten()) # 확률 분포로 정규화

        sampled = np.random.choice(len(p), size=1, p=p) # 확률에 따라 다음 단어 선택
        if (skip_ids is None) or (sampled not in skip_ids): # 제외 단어가 아니면
            x = sampled # 샘플링된 단어를 다음 입력으로 설정
            word_ids.append(int(x)) # 결과 리스트에 추가

    return word_ids # 생성된 단어 ID 시퀀스 반환
```

요약: 시작 단어부터 확률적 샘플링으로 한 단어씩 생성하며, 이전 출력이 다음 입력이 되는 자기회귀적 테스트 생성 방식